Java Programming Fundamentals

Classes, control flow, and collections

A concise, original learning guide with practical examples and a mini project.

Designed for beginners • 3 learning pages + cover

1. Java Basics

Java is a general-purpose, object-oriented programming language used in enterprise systems, Android development, backend services, and many educational courses.

First program

```
public class Main {\n    public static void main(String[] args) {\n        System.out.println("Hello, w
orld!");\n    }\n}
```

Java source code is compiled to bytecode and runs on a Java Virtual Machine (JVM). The public class name normally matches the source filename.

Variables and types

```
int quantity = 3;\ndouble price = 49.50;\nboolean available = true;\nString title = "Python Basics";
```

Java is statically typed. Primitive types include int, double, boolean, and char. String is a reference type used for text.

Conditions and loops

```
if (quantity > 0) {\n    System.out.println("In stock");\n}\nfor (int i = 1; i <= 3; i++) {\n    System
.out.println(i);\n}
```

2. Classes and Objects

Define a class

```
class Book {\n    private String title;\n\n    public Book(String title) {\n        this.title = title;
\n    }\n\n    public String getTitle() {\n        return title;\n    }\n}
```

A class defines fields and behavior. An object is an instance of a class. Encapsulation keeps data private and exposes controlled methods.

Use an object

```
Book book = new Book("Java Basics");\nSystem.out.println(book.getTitle());
```

Constructors initialize new objects. Use meaningful names and keep methods focused. Add validation in constructors when invalid objects should not be created.

Collections

```
import java.util.ArrayList;\nimport java.util.List;\n\nList<String> languages = new ArrayList<>();\nlan
guages.add("Java");\nlanguages.add("Python");\nSystem.out.println(languages.size());
```

Collections store groups of values. Generics such as List<String> specify the expected element type and help prevent mistakes.

3. Exceptions and Good Practices

Handling exceptions

```
try {\n    int result = 10 / 0;\n    System.out.println(result);\n} catch (ArithmeticException error) {
\n    System.out.println("Cannot divide by zero");\n}
```

Exceptions represent exceptional conditions. Catch errors you can handle, and avoid swallowing exceptions without logging or a meaningful response.

Methods and testing

Extract repeated logic into methods. Use unit tests to check normal inputs, edge cases, and failure conditions. Keep classes cohesive and avoid unnecessary complexity.

Mini project

Build a console-based library catalogue with a Book class, an ArrayList<Book>, and methods to add, search, list, and remove books.

Learn the fundamentals first, then explore interfaces, inheritance, streams, generics, and build tools such as Maven or Gradle.